

1. Sine wave

```
/// I have implemented the following features:
// - implemented objects such as confetti, boxes, and camera
// - implemented noise to create a random movement of the confetti
// - implemented the sine wave that also includes noise to create a some random movement of the
boxes
// - implemented a point light to create a light effect on the boxes when they are high enough
// - reduced the number of boxes to improve the performance

var confetti;
var boxes;
var myCamera;
function setup() {
  createCanvas(900, 600, WEBGL);
  angleMode(DEGREES)

  // CRATING AND INITATING OBJECTS
  confetti = new Confetti()
  boxes = new Boxes()
  myCamera = new Camera()
}

function draw() {
  background(0);
  ambientLight(255)

  // I wrote this code
  confetti.update()
  confetti.draw()

  boxes.draw()

  myCamera.flyAround()
  // end of code I wrote
}

// I wrote this code
class Camera{
  // Camera class - contains static and fly around camera
  constructor(){}

  static(){
    // This code uses static camera instead of the fly around one - I used this for
    developing this code
    camera(0,-500,800)
  }
  flyAround(){
    // Fly around camera - used sin and cos function as per instruction
    let locX = sin(frameCount/2)*800
    let locZ = cos(frameCount/2)*800
    camera(locX,-500,locZ)
  }
}

class Confetti{
  // Confettin class - contains all functionality of the confetti
  constructor(){
    // Constructs the confetti object
    this.confLocs =[] // The array containing location of individula confetti
    this.confTheta =[] // The array containing the rotation angle theta of each confetti
  }
  leaf
  this.setup() // Sets up the confetti
  this.size = 25 // The size of the single confetti plane
}
  setup(){
```

```

    for (let i=0;i<200;i++){
        // Single confetti location - uses random function to generate initial location
        let rX = int(random(-500,500))
        let rY = int(random(-800,0))
        let rZ = int(random(-500,500))
        let vec3d = createVector(rX, rY, rZ) // Single confetti location is stored in 3D
vector
        this.confLocs.push(vec3d) // Pushing the vector into the locs array

        // Single confetti angle
        this.confTheta.push(int(random(0,360))) // The angle is randomly generated and
        pushed immediately to the array
    }
}
draw(){
    // Method responsible for drawing the all confetti. It uses the location and theta angle
arrays.
    for (let i=0;i<this.confLocs.length;i++){
        // Creating single confetti
        push()
        translate(this.confLocs[i])
        let nY = noise(i, frameCount/50) // Using Perlin noise for the movement in Y
axis

        let rY = map(nY, 0, 1, 0, 360) // Mapping noise to have meaningful value range
        rotateX(this.confTheta[i])
        rotateY(rY)
        noStroke()
        let c = map(i, 0, this.confLocs.length, 0, 255) // Using the number of the
        confettin to influence the colour of individual confetti
        ambientMaterial(c,255-c,0)
        plane(this.size, this.size)
        pop()
    }
}

update(){
    // Method responsible for updating the position and the rotation of each confetti plane
    for(let i=0;i<this.confLocs.length;i++){
        // Updating single confetti vertical location and rotation
        // this.confLocs[i].y++
        let nX = noise(i, frameCount/30)
        let deltaX = map(nX, 0, 1, -2, 2)
        let nY = noise(i, frameCount/50)
        this.confLocs[i].y+=nY*3
        this.confLocs[i].x+=deltaX
        this.confTheta[i] +=10

        // Repositioning of the single confetti if it reaches the bottom level
        if (this.confLocs[i].y>0){
            this.confLocs[i].y =-800
        }
    }
}
}

```

```

class Boxes{
    // This class contain all the code responsible for generating and drawing the boxes
    constructor(){
        // Constructing the boxes object
        this.boxSize = 75
    }

    draw(){
        // All boxes are generated and drawn by this method
        for(let i=-400; i<=400;i+=this.boxSize){

```

```

        for(let j=-400; j<=400;j+=this.boxSize){
            // The 2D array of boxes is being generated here by neste loop as per
instruction
            let distance = dist(0,0,0,i,0,j)          // i and j represnt the position of
each box.
            let nF = noise(i/300,j/300,frameCount/50) // Introducing Perlin noise
            let length = (sin(distance/2-nF*600- frameCount*3)+2)*50 // Generating sine wave
with addition of the noise so the movement is not so uniform

            push()
            translate(i,0,j)
            if (length>130){ // Implementing the point light - it is being switched on
whenever the length of the box exceed given threshold
                pointLight(255, 255, 255, i, -400, j)
            }

            // normalMaterial()
            let cR = map(length, 0, 200, 0, 255)
            ambientMaterial(cR/2,0,255-cR/2) // A box colour depends on its lenght

            stroke(0)
            strokeWeight(2)
            box(this.boxSize,length,this.boxSize)

            pop()
        }
    }
}

// end of code I wrote

```

2. The Average Face

```
var imgs = [];
var avgImg;
var numOfImages = 30;

// My variables
var img;
var averageFace;
var canvasWidth;
var canvasHeight;
var transitImg;
var timeout;

////////////////////////////////////
function preload() { // preload() runs once
  // I wrote this code
  for (let i = 0; i<30;i++){
    // Iterating over the images in the assets folder pushing preloaded images into the imgs
    array
    let filename = str(i) + ".jpg"
    let folder = "assets/"
    let path = folder + filename
    let img = loadImage(path)
    // console.log(path)
    imgs.push(img)

  }
  console.log('---images preloaded---')
  // end of the code I wrote
}

////////////////////////////////////
function setup() {

  // I wrote this code

  noStroke()
  // Setting the size of the canvas to fit two images next to each other - first image is used
  to get the size
  canvasWidth = imgs[0].width*2
  canvasHeight = imgs[0].height

  createCanvas(canvasWidth, canvasHeight );
  pixelDensity(1);

  // Creating the offset buffers for average and transit images
  avgImg = createGraphics(canvasWidth/2,canvasHeight)
  transitImg = createGraphics(canvasWidth/2,canvasHeight)

  // instance of the AverageFace class is created
  averageFace = new AverageFace();

  // end of the code I wrote
}

////////////////////////////////////
function draw() {
  background(125);

  // I wrote this code

  // LOADING PIXELS
  averageFace.loadPixelsOfAllImagsAndAvgImg()

  // LEFT IMAGE
  averageFace.drawRandomImage()

  // UPDATING PIXELS
  averageFace.updateAvgImgForAverageOfAllImages()
}
```

```

    // RIGHT IMAGE
    averageFace.drawRightImage()

    noLoop()

    // end of the code I wrote
}

// I wrote this code
function keyPressed(){
    // Key press handler for the averageFace object
    averageFace.keyPressedHandler()
}

function mouseMoved(){
    // Mouse moved handler for the average face object
    averageFace.mouseMovedHandler()
}

class AverageFace{
    // Class contains all the codee
    constructor(){

        this.firstImage = imgs[0]
        this.currentImgIndex = int(random(0,imgs.length-1)) // Random image index
        this.transition = 100 // Range 0-100, indicates transition between LEFT and RIGHT IMAGE
        // Flags
        this.newImageLoaded = true
        this.readyToDrawAllImages = false
    }

    drawRandomImage(){
        // Function draws the random image when the space is pressed
        let currentImg = imgs[this.currentImgIndex]
        image(currentImg,0,0,currentImg.width,currentImg.height)
    }

    loadPixelsOfAllImagsAndAvgImg(){
        // This fuction is called whe A is pressed - all images are displayed which were used
        // to generate the average image - this is simple extension
        avgImg.loadPixels()
        transitImg.loadPixels();

        for (let i=0;i<imgs.length;i++){
            imgs[i].loadPixels()
        }
    }

    updateAvgImgForAverageOfAllImages(){
        for (let y=0;y<this.firstImage.height;y++){ // Moving along y-axis
            for (let x=0;x<this.firstImage.width;x++){ // Moving along x-axis
                let index = ((y*this.firstImage.width)+x)*4; // Index of the current pixel

                ///// Calculating average value for each pixel in avgImg from all images

                let sumR =0
                let sumG =0
                let sumB =0
                let sumA =0

                for (let i=0;i<imgs.length;i++){ // Iterating through all of the images
                    sumR += imgs[i].pixels[index+0]; // Red
                    sumG += imgs[i].pixels[index+1]; // Green
                    sumB += imgs[i].pixels[index+2]; // Blue

```

```

        sumA += imgs[i].pixels[index+3];          // Alpha
    }
    // Calculating averages for each pixel
    let avgR = sumR/imgs.length
    let avgG = sumG/imgs.length
    let avgB = sumB/imgs.length
    let avgA = sumA/imgs.length

    //// Updating avgImg pixels
    avgImg.pixels[index] = avgR
    avgImg.pixels[index+1] = avgG
    avgImg.pixels[index+2] = avgB
    avgImg.pixels[index+3] = avgA
    }
}

avgImg.updatePixels()
}

drawRightImage(){
    if (this.newImageLoaded){ // Loads avgImg if mouse haven't been moved since the program
started
        this.newImageLoaded=false
        image(avgImg,canvasWidth/2,0, avgImg.width,avgImg.height)

        /// Show instructions
        this.#drawInstructions()

    }
    else if(this.readyToDrawAllImages){
        this.#drawAllImages()
        this.readyToDrawAllImages = false
    }
    else{ // Whenever the mouse was moved the transitImg is drawn
        image(transitImg,canvasWidth/2,0,canvasWidth/2,height)
    }
}

/// PRIVATE METHODS //////////////////////////////////////
#updateRandomImageIndex(){
    // Generates the random index to be used to display the initial image
    this.currentImgIndex = int(random(0,imgs.length-1))
    console.log(`Displaying image ${this.currentImgIndex}`)
    loop()
}

#transitBetweenLeftAndRightImage(){
    // Code responsible for generating the transition between the initial image and the
average image
    let mT = map(mouseX,0,width,0,1)
    this.transition = constrain(mT, 0, 1)

    for (let y=0;y<this.firstImage.height;y++){ // Moving along y-axis
        for (let x=0;x<this.firstImage.width;x++){ // Moving along x-axis
            let pixelIndex= ((y*this.firstImage.width)+x)*4; // Index of the current
pixel

            // Iteraing through pixel
            for(let i=0;i<4;i++){
                let left = imgs[this.currentImgIndex].pixels[pixelIndex+i]
                let right = avgImg.pixels[pixelIndex+i]

                let lerpVal = lerp(left,right, this.transition)
                transitImg.pixels[pixelIndex+i] = lerpVal
            }
        }
    }
}

```

```

    }
    transitImg.updatePixels()
    this.newImageLoaded = false
    loop()
  }

  #drawInstructions(){
    // Draws instruction text on the canvas at the initial state of the program
    push()
    stroke(255)
    strokeWeight(this.firstImage.width/100)
    textSize(this.firstImage.width/13)
    text("- Press 'A' to see all images. \n- Move the mouse left or right (within the
canvas area)\n to see the transition.\n- Press any key to change the image", 50, 50,)
    pop()
  }

  #drawAllImages(){
    // Draws all images instead of one
    let widthNum = 10
    let heightNum = 30/widthNum
    let imgSize = width/widthNum

    background(125)
    for (let i =0; i<heightNum;i++){
      for (let j=0; j<widthNum;j++){
        image(imgs[i*heightNum+j],j*imgSize,i*imgSize,imgSize,imgSize)
      }
    }
  }

  /// EVENT HANDLERS //////////////////////////////////////
  keyPressedHandler(){
    if (keyPressed && (key === "A" || key==="a")){
      this.readyToDrawAllImages = true
      loop()
    }
    else if (keyPressed){
      this.#updateRandomImageIndex()
      this.#transitBetweenLeftAndRightImage()
      // this.drawAvgImg();
    }
  }
  mouseMovedHandler(){
    // The RIGHT image is redrawn only every 30th pixel crossed by the mouse or after 300 ms
    after mouse stopped moving
    if (mouseMoved && mouseX%20==0){ // Every 30th pixel block
      this.#transitBetweenLeftAndRightImage()
    }
    else{// Time out block
      clearTimeout(timeout);
      timeout = setTimeout(() => { this.#transitBetweenLeftAndRightImage(); }, 300);
      // REFERENCE https://developer.mozilla.org/en-US/docs/Web/API/setTimeout
    }
  }
}
// end of the code I wrote

```

3. The Instagram Filter

```
// Image of Husky Creative commons from Wikipedia:
// https://en.wikipedia.org/wiki/Dog#/media/File:Siberian_Husky_pho.jpg

// INTRODUCTION: I extended the functionality of this assignment by introducing buttons which
// enable to turn of or on each filter separately
var centerX;
var centerY;
var imgIn;
// I wrote this code
var buffer;
var buttonBlur;
var buttonVignetting;
var buttonBorder;
var buttonSepia;
var buttonsState={
  "sepia": true,
  "vignetting": true,
  "blur": true,
  "border":true
}

// end of the code I wrote

var matrix = [

  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64],
  [1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64, 1/64]
];

////////////////////////////////////
function preload() {
  imgIn = loadImage("assets/husky.jpg");
}
////////////////////////////////////
function setup() {
  createCanvas((imgIn.width * 2), imgIn.height);
  // I wrote this code
  buffer = createGraphics(imgIn.width, imgIn.height)

  // Creating instances of myButton
  buttonBlur = new myButton("blur", 20,40)
  buttonSepia = new myButton("sepia", 20,70);
  buttonVignetting = new myButton("vignetting",20,100);
  buttonBorder = new myButton("border",20,130)

  // end of the code I wrote
}
////////////////////////////////////
function draw() {
  background(125);
  image(imgIn, 0, 0);
  image(earlyBirdFilter(imgIn), imgIn.width, 0);
  noLoop();
}
////////////////////////////////////
function mousePressed(){
  loop();
}
////////////////////////////////////
function earlyBirdFilter(img){
  var resultImg = createImage(imgIn.width, imgIn.height);
```



```

// I wrote this code
centerX = floor(img.width / 2)
centerY = floor(img.height / 2)
resultImg = imgIn

// SEPIA FILTER
if (buttonSepia.state) {
  resultImg = sepiaFilter(imgIn);
}

// VIGNETTING FILTER
if (buttonVignetting.state){
  resultImg = darkCorners(resultImg);
}
// BLUR FILTER
if (buttonBlur.state){
  resultImg = radialBlurFilter(resultImg);
}

// BORDER FILTER
if (buttonBorder.state){
  resultImg = borderFilter(resultImg)
}

// end of the code I wrote
return resultImg;
}
// I wrote this code
function sepiaFilter(img){
  // Sepia filter code - modifies each channel to give the sepia effect
  imgOut = createImage(img.width,img.height)

  img.loadPixels();
  imgOut.loadPixels()

  for(let x=0;x<img.width;x++){
    for(let y=0;y<img.height;y++){
      // Traversing the pixels array

      let index = (img.width*y+x)*4;

      // Getting the each channel values
      let oldRed = img.pixels[index + 0]
      let oldGreen = img.pixels[index + 1]
      let oldBlue = img.pixels[index + 2]

      // Creating new values of each channel based on the original values
      newRed = (oldRed * .393) + (oldGreen * .769) + (oldBlue * .189)
      newGreen = (oldRed * .349) + (oldGreen * .686) + (oldBlue * .168)
      newBlue = (oldRed * .272) + (oldGreen * .534) + (oldBlue * .131)

      // Modifying the pixels array
      imgOut.pixels[index+0] = newRed;
      imgOut.pixels[index+1] = newGreen;
      imgOut.pixels[index+2] = newBlue;
      imgOut.pixels[index+3] = 255;
    }
  }

  // Letting know the p5.js that the pixels were modified
  imgOut.updatePixels()
  return imgOut
}

function darkCorners(img){
  // The function that gives the vignetting effect
  for(let x=0;x<img.width;x++){
    for(let y=0;y<img.height;y++){

```

```

    // Traversing the pixels array
    let index = (img.width*y+x)*4;

    // Getting each colour of the pixel
    let oldRed = img.pixels[index + 0]
    let oldGreen = img.pixels[index + 1]
    let oldBlue = img.pixels[index + 2]

    // Measuring the distance from the center
    let d = dist(centerX,centerY,x,y)
    let dynLum =0;

    // Reduces the intensity of each colour depending on the distance from the center

    if (d<=300){ // No change in intensity
        dynLum = 1;
    }
    else if (d<=450){ // Gradual reduction of the intensity
        dynLum = map(d,300,450,1,0.4)
    }
    else
    {
        dynLum = map(d,450,600,0.4,0) // Reduction of the intensity to total darkness
    }

    // Modifying pixels
    imgOut.pixels[index+0] = oldRed*dynLum;
    imgOut.pixels[index+1] = oldGreen*dynLum;
    imgOut.pixels[index+2] = oldBlue*dynLum;
    imgOut.pixels[index+3] = 255;
}
}
// Letting know p5.js that the imgOut pixel array was modified
imgOut.updatePixels()
return imgOut
}

function radialBlurFilter(img){
    // Radial blur is using convolution to blur the image. It need to use kernel - the matrix at
    the beginning of the code to do the calculations
    var imgOut = createImage(img.width, img.height)
    var matrixSize = matrix.length;

    // Loading pixels to gain direct access to them
    imgOut.loadPixels()
    img.loadPixels()

    for(var x=0;x<img.width;x++){
        for(var y=0;y<img.height;y++){

            // Traversing pixels array
            let index = (y*img.width + x) * 4

            let r = img.pixels[index + 0]
            let g = img.pixels[index + 1]
            let b = img.pixels[index + 2]

            // Calling convolution function to blur the image
            var c = this.convolution (x, y, matrix, matrixSize, img)

            // Blur intensity depends on the distance from the last click of the mouse
            var d =dist(x,y,mouseX-img.width,mouseY)

            var dynBlur = map(d,100,300,0,1)
            dynBlur = constrain(dynBlur,0,1)
            imgOut.pixels[index + 0] = c[0]*dynBlur + r*(1-dynBlur);
            imgOut.pixels[index + 1] = c[1]*dynBlur + g*(1-dynBlur);
            imgOut.pixels[index + 2] = c[2]*dynBlur + b*(1-dynBlur);
            imgOut.pixels[index + 3] = 255;

```

```

    }
  }
  imgOut.updatePixels();
  return imgOut;
}

function convolution(x, y, matrix, matrixSize, img){
  // The convolution function as per instruction
  var totalRed = 0;
  var totalGreen = 0;
  var totalBlue = 0;

  var offset = floor(matrixSize/2);

  for (var i=0; i<matrixSize;i++){ //
    for(var j=0; j<matrixSize;j++){
      var xloc = x + i - offset;
      var yloc = y + j - offset;
      var index = (img.width*yloc+xloc)*4;
      index = constrain(index, 0, img.pixels.length-1);

      totalRed +=img.pixels[index +0] * matrix[i][j]
      totalGreen+=img.pixels[index +1] * matrix[i][j]
      totalBlue +=img.pixels[index +2] * matrix[i][j]

    }
  }
  return [totalRed,totalGreen, totalBlue]
}

function borderFilter(img){
  buffer.image(img,0,0)
  buffer.noFill()
  buffer.strokeWeight(20)
  buffer.stroke(255)
  buffer.rect(0,0,img.width,img.height, 50) // Rounded frame
  buffer.rect(0,0,img.width,img.height) // White background to eliminate the black corners

  return buffer;
}

class myButton{
  constructor(filterName, posX, posY){
    this.filterName = filterName
    this.posX = posX;
    this.posY = posY;

    this.state = true

    this.colOn = color(25, 223, 50, 50);
    this.colOff = color(225, 23, 50, 50);

    // Creating button
    this.btn = createButton(this.filterName)
    this.btn.position(this.posX, this.posY)

    // Styling the button
    this.btn.style('background-color', this.colOn)
    this.btn.style('font-size', '18px')
    this.btn.style('border-radius', '5px')

    // Button mousePressed functionality
    this.btn.mousePressed(() => {
      this.changeState(filterName)
    })
  }

  changeState(filterName){
    this.state = !this.state
  }
}

```

```
    if (this.state){
      this.btn.style('background-color', this.colOn)
    }
    else {
      this.btn.style('background-color', this.colOff)
    }
  }
}
// end of the code I wrote
```

4. The Webcam Piano

sketch.js

```
var video;
var threshold = 20;
var thresholdSlider;
var button;
var prevImg;
var currImg;
var diffImg;
var grid;

// MY CODE
// All modifications to my code were implemented in the Grid.js file. The only modification in
this file is the scale factor below.
// Additionally, I had to download newer versions of the p5.sound and p5.sound.min files, as the
older versions contained a bug that prevented sound from being generated correctly.
// My apologies for terrible sound effects . I was doing my best ; - )
var vScale = 1 // Using scale to reduce the size of the video due to performance of my machine =
for debugging only

function setup() {
  createCanvas(640*2*vScale, 480*vScale);
  pixelDensity(1)
  video = createCapture(VIDEO)
  video.hide()
  noStroke()

  thresholdSlider = createSlider(0, 255, 20)
  thresholdSlider.position(20, 20)
  // I wrote this code
  grid = new Grid(640*vScale, 480*vScale);
  monoSynth = new p5.MonoSynth();

  // end of the code I wrote
}

function draw() {
  background(0)
  image(video, 0,0, 640*vScale, 480*vScale)

  currImg = createImage(video.width, video.height)
  currImg.copy(video,
    0,0, video.width, video.height,
    0,0, video.width, video.height)

  let scaleFactor = 2
  // scaleFactor = 8 // FOR TESTING ONLY - due to performace of my local machine
  currImg.filter(BLUR,3)
  currImg.resize(currImg.width/scaleFactor, currImg.height/scaleFactor)

  diffImg = createImage(video.width, video.height)
  diffImg.resize(diffImg.width/scaleFactor,diffImg.height/scaleFactor)
  diffImg.loadPixels()

  threshold = thresholdSlider.value()

  if (prevImg){ // We only perform the loop if the prevImg exists
    prevImg.loadPixels()
    currImg.loadPixels()
    for (var x = 0; x < video.width; x++) {
      for (var y = 0; y < video.height; y++) {
        var index = (y * video.width + x) * 4;

        // Here is where all the computer vision happens
        var redSource = currImg.pixels[index + 0]
        var greenSource = currImg.pixels[index + 1]
        var blueSource = currImg.pixels[index + 2]
```

```

        var redBack = prevImg.pixels[index + 0]
        var greenBack = prevImg.pixels[index + 1]
        var blueBack = prevImg.pixels[index + 2]

        var d = dist(redSource, greenSource, blueSource,
            redBack, greenBack, blueBack)

        if (d > threshold) {
            diffImg.pixels[index + 0] = 0;
            diffImg.pixels[index + 1] = 0;
            diffImg.pixels[index + 2] = 0;
            diffImg.pixels[index + 3] = 255;
        }
        else {
            diffImg.pixels[index + 0] = 255;
            diffImg.pixels[index + 1] = 255;
            diffImg.pixels[index + 2] = 255;
            diffImg.pixels[index + 3] = 255;
        }
    }
}

diffImg.updatePixels()
image(diffImg, 640*vScale, 0, 640*vScale, 480*vScale)

// Storing current image to be used as previous frame in the next draw iteration
prevImg = createImage(currImg.width, currImg.height)
prevImg.copy(currImg,
    0, 0, currImg.width, currImg.height,
    0, 0, currImg.width, currImg.height)

grid.run(diffImg)
}

```

grid.js

```

class Grid {
    //////////////////////////////////////
    constructor(_w, _h) {
        this.gridWidth = _w;
        this.gridHeight = _h;
        this.noteSize = 40;
        this.notePos = [];
        this.noteState = [];

        // I wrote this code
        this.polySynth = new p5.PolySynth(); // Creating instances of Poly- and MonoSynth to be able to
        synthesize the sound. polySynth object required the latest latest version of p5.sound and p5.sound.min to
        function correctly.
        this.monoSynth = new p5.MonoSynth()

        this.noteInitiated = []; // The array that indicates if the sufficient motion in the area appeared to
        initiate the sound generation - it stores boolean values. The value is true only when the motion was
        detected where previously was not present.
        this.notesArr = []; // The array that contain notes names e.g. A5, C7. It is populated by
        generateNotesArr() method. It is being used by polySynth object to generate the sound.
        this.generateNotesArr(); // The function mentioned above.
        // end of code I wrote

        // initialise grid structure and state
        for (var x = 0; x < _w; x += this.noteSize) {
            var posColumn = [];
            var stateColumn = [];
            var noteColumn = []; // MY CODE - temporary array used to populate noteInitiated array below

            for (var y = 0; y < _h; y += this.noteSize) {
                posColumn.push(createVector(x + this.noteSize / 2, y + this.noteSize / 2));
                stateColumn.push(0);
                noteColumn.push(false) // MY CODE - each column contains initially false values only
            }
            this.notePos.push(posColumn);
            this.noteState.push(stateColumn);

```

```

        this.noteInitiated.push(noteColumn) //MY CODE - noteInitiated array is being populated by false value.
    }
}
////////////////////////////////////
run(img) {
    img.loadPixels();
    this.findActiveNotes(img);
    this.drawActiveNotes(img);

    this.playActiveNote(); //MY CODE - runs the logic responsible for generating sound
}
////////////////////////////////////
drawActiveNotes(img) {
    // draw active notes
    fill(255);
    noStroke();
    for (var i = 0; i < this.notePos.length; i++) {
        for (var j = 0; j < this.notePos[i].length; j++) {
            var x = this.notePos[i][j].x;
            var y = this.notePos[i][j].y;
            if (this.noteState[i][j] > 0) {
                var alpha = this.noteState[i][j] * 200;
                var c1 = color(255, 0, 0, alpha);
                var c2 = color(0, 255, 0, alpha);
                var mix = lerpColor(c1, c2, map(i, 0, this.notePos.length, 0, 1));
                fill(mix);
                var s = this.noteState[i][j];
                ellipse(x, y, this.noteSize * s, this.noteSize * s);
            }
            this.noteState[i][j] -= 0.25;
            this.noteState[i][j] = constrain(this.noteState[i][j], 0, 1);
        }
    }
}
////////////////////////////////////
findActiveNotes(img) {
    for (var x = 0; x < img.width; x += 1) {
        for (var y = 0; y < img.height; y += 1) {
            var index = (x + (y * img.width)) * 4;
            var state = img.pixels[index + 0];
            if (state == 0) { // if pixel is black (ie there is movement)
                // find which note to activate
                var screenX = map(x, 0, img.width, 0, this.gridWidth);
                var screenY = map(y, 0, img.height, 0, this.gridHeight);
                var i = int(screenX / this.noteSize);
                var j = int(screenY / this.noteSize);
                this.noteState[i][j] = 1;
            }
        }
    }
}

// I wrote this code
playActiveNote() {
    // Method responsible for calling playSynth and showNote function
    for (var i = 0; i < this.notePos.length; i++) {
        for (var j = 0; j < this.notePos[i].length; j++) {
            var x = this.notePos[i][j].x;
            var y = this.notePos[i][j].y;
            // Until this point nested loop is the same as in the drawActiveNotes method
            var s = this.noteState[i][j];
            if (s > 0.7) { // The area will be activated only if the motion is sufficiently large

                if (this.noteInitiated[i][j] == false) {
                    // The both function will be generated only if the area was not active in the previous frame
                    this.noteInitiated[i][j] = true
                    this.playSynth(i, j)
                    this.showNote(x, y, i, j, s)
                }
            }
            else {
                this.noteInitiated[i][j] = false
            }
        }
    }
}
playSynth(i,j) {
    userStartAudio();
    // This function was created based on https://p5js.org/reference/#/p5.PolySynth

```

```

    let index = (i*this.notePos.length+j) // The row and column of the areas is converted to the index to be
used in recognising which note should be played

    // console.log("PLAYING SOUND", this.notesArr[index%(this.notesArr.length-1)])

    let note = this.notesArr[index%(this.notesArr.length-1)] // The note is selected from the notesArr which
contain the proper note names. The notes will repeat themselves on the canvas

    // note velocity (volume, from 0 to 1)
    let velocity = 0.5;
    // time from now (in seconds)
    let time = 0;
    // note duration (in seconds)
    let dur = random(0.3,1);

    // GENERATING SOUND
    this.polySynth.play(note, velocity, time, dur);
}

showNote(x,y,i,j,s) {
    // This code draws a border around the areas to indicate which one is the source of the sound. It also
indicates which note is being generated. One are can only generate sound once while being active. It has to
become inactive to be able to generate sound again.

    push()
    // Drawing border
    let c = color(i * 40, 0, j * 40)
    stroke(c)
    strokeWeight(5)
    ellipse(x, y, this.noteSize * s * 1.2, this.noteSize * s * 1.2);

    // Drawing text
    let index = (i*this.notePos.length+j)
    noStroke()
    fill(c)
    textSize(20)
    text(str(this.notesArr[index%(this.notesArr.length-1)]),x-10,y+10)
    pop()
}

generateNotesArr(){
    // Function runs once - it populates the notesArr with correct note names
    let letters = ["A","B","C","D","E","F","G"]
    for(let k=0;k<letters.length;k++) {
        for(let m=1;m<=8;m++){
            this.notesArr.push(letters[k]+str(m))
        }
    }
}
}
}

```

// end of the code I wrote